ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÁO CÁO BÀI TẬP LỚN

Hệ thống Nhận diện dáng đi

Học phần: Nhập môn Học máy và Khai phá dữ liệu

Nhóm Sinh viên thực hiện

Họ và tên	MSSV
Bàng Tiến Thành	20225669
Lục Minh Đức	20225810
Trương Công Thái Đức	20225815
Dương Đức Duy	20225828
Nguyễn Mạnh Tuấn	20225679

Hà Nội, tháng 1 năm 2025

Mục lục

Tóm tắt **4**

1 Giới thiệu **5**

1.1 Lý do chọn đề tài 5

1.2 Mục tiêu đề tài 5

1.3 Phạm vi nghiên cứu 6

2 Cơ sở lý thuyết **7**

2.1 Trí tuệ nhân tạo và Học sâu 7

2.1.1 Học sâu (Deep Learning) 7

2.2 Mạng nơ-ron tích chập (CNN) 8

2.2.1 Tổng quan về CNN 8

2.2.2 Tại sao CNN vượt trội trong xử lý ảnh? 8

2.2.3 Cơ chế trích xuất đặc trưng phân tầng 9

2.2.4 Các thành phần của CNN 9

2.2.5 Vì sao CNN lại hiệu quả trong nhận diện ảnh? 14

2.3 Hàm mất mát Cross-Entropy 14

2.3.1 Cơ sở lý thuyết 15

2.3.2 Công thức toán học 15

2.3.3 Trường hợp phân loại nhị phân (Binary Cross-Entropy) 15

2.4 Thuật toán tối ưu Adam 16

2.4.1 Cơ chế hoạt động 16

2.4.2 Lý do lựa chọn Adam 17

2.5 Hàm kích hoạt Softmax 17

2.5.1 Định nghĩa toán học 18

2.5.2 Tính chất quan trọng 18

2.5.3 Ứng dụng trong đồ án 19

3 Dataset và tiền xử lý dữ liệu **20**
3.1 Dataset sử dụng 20
3.2 Tiền xử lý dữ liệu 20

4 Xây dựng mô hình **21**
4.1 Kiến trúc mô hình 21
4.1.1 Ý tưởng thiết kế 21
4.1.2 Đoạn code định nghĩa CNN 21
4.1.3 Kích thước tensor qua các tầng 23
4.2 Huấn luyện mô hình 23
4.2.1 Thiết lập huấn luyện (Loss, Optimizer, Epoch) . . . 23
4.2.2 Thuật toán huấn luyện và validation 24
4.2.3 Quy trình huấn luyện và đánh giá (Training & Validation Loop) 25

5 Đánh giá kết quả **28**
5.1 Độ chính xác 28
5.1.1 Accuracy tổng thể 28
5.1.2 Accuracy theo lớp (quan trọng khi lệch lớp) 28
5.1.3 Đoạn code đánh giá 29
5.1.4 Độ chính xác của mô hình 30
5.2 Nhận xét và đánh giá chi tiết 30
5.2.1 Về chiến lược xử lý dữ liệu 30
5.2.2 Về kiến trúc mô hình CNN 31
5.2.3 Về khả năng ứng dụng thực tế 31

6 Xây dựng giao diện Web **33**
6.1 Công nghệ sử dụng 33
6.2 Luồng hoạt động 33
6.3 Mã nguồn của giao diện web 34
6.4 Đánh giá và Nhận xét 35
6.4.1 Về trải nghiệm người dùng (UX) 35
6.4.2 Về khía cạnh kỹ thuật 36
6.4.3 Hạn chế cần khắc phục 36

7 Hạn chế và hướng phát triển **37**
7.1 Hạn chế 37
7.2 Hướng phát triển 38

8 Kết luận và Hướng phát triển **40**
8.1 Tổng kết các kết quả đạt được 40
8.2 Đánh giá tổng quan 41
8.3 Hướng phát triển trong tương lai 41
8.4 Lời kết 42

Tài liệu tham khảo **43**

Tóm tắt

Báo cáo này trình bày chi tiết quá trình nghiên cứu, thiết kế và xây dựng một hệ thống nhận diện tình trạng hói đầu từ ảnh khuôn mặt dựa trên các phương pháp học sâu (Deep Learning). Bài toán được mô hình hoá dưới dạng phân loại ảnh nhị phân, trong đó mỗi ảnh đầu vào được gán vào một trong hai lớp: Bald (hói) và NotBald (không hói). Mục tiêu của hệ thống là tự động nhận biết tình trạng hói đầu dựa trên đặc trưng hình ảnh khuôn mặt, góp phần minh họa khả năng ứng dụng của học sâu trong bài toán thị giác máy tính.

Hệ thống được xây dựng dựa trên mạng nơ-ron tích chập (Convolutional Neural Network – CNN), đóng vai trò trích xuất đặc trưng không gian từ ảnh đầu vào và thực hiện dự đoán nhãn. Trong quá trình huấn luyện, dữ liệu ảnh được tiền xử lý nhằm chuẩn hóa kích thước, giá trị điểm ảnh và tăng tính ổn định của mô hình. Hàm mất mát Cross-Entropy Loss được sử dụng để đo lường sai lệch giữa dự đoán của mô hình và nhãn thực tế, trong khi thuật toán tối ưu Adam Optimizer được áp dụng nhằm cải thiện tốc độ hội tụ và hiệu quả huấn luyện.

Phần đánh giá mô hình tập trung vào chỉ số độ chính xác tổng thể (overall accuracy) và độ chính xác theo từng lớp (class-wise accuracy), từ đó phân tích khả năng nhận diện của mô hình đối với từng nhóm đối tượng. Việc đánh giá theo từng lớp giúp phát hiện nguy cơ mất cân bằng dữ liệu (class imbalance) và hạn chế hiện tượng mô hình thiên lệch về một lớp cụ thể. Kết quả thực nghiệm cho thấy mô hình CNN có khả năng học được các đặc trưng liên quan đến tình trạng hói đầu và đạt độ chính xác khả quan trên tập dữ liệu kiểm thử.

Bên cạnh phần mô hình học sâu, báo cáo cũng trình bày mô tả luồng hoạt động của giao diện web ở mức tổng quan, bao gồm quá trình tải ảnh, xử lý ảnh đầu vào và hiển thị kết quả dự đoán. Phần giao diện được xây dựng nhằm minh họa khả năng ứng dụng thực tế của hệ thống, đồng thời được giới hạn ở mức mô tả chức năng, phù hợp với phạm vi và yêu cầu của đồ án học phần.

Chương 1

Giới thiệu

1.1 Lý do chọn đề tài

Trong bối cảnh trí tuệ nhân tạo và thị giác máy tính ngày càng phát triển, các hệ thống nhận diện con người đang được ứng dụng rộng rãi trong nhiều lĩnh vực, đặc biệt là an ninh và giám sát. Bên cạnh các phương pháp truyền thống như nhận diện khuôn mặt hay vân tay, nhận diện dáng đi là một hướng tiếp cận mới, cho phép xác định con người từ xa mà không cần sự hợp tác trực tiếp, phù hợp với các môi trường giám sát thực tế.

Việc lựa chọn đề tài Hệ thống Nhận diện Dáng đi giúp nhóm vận dụng các kiến thức đã học về xử lý ảnh, học máy và trí tuệ nhân tạo để giải quyết một bài toán thực tế. Thông qua đồ án, nhóm có cơ hội rèn luyện kỹ năng phân tích, thiết kế và triển khai hệ thống, đồng thời nâng cao khả năng nghiên cứu và ứng dụng công nghệ vào thực tiễn.

1.2 Mục tiêu đề tài

- Xây dựng hệ thống nhận dạng cá nhân dựa trên dáng đi trong bối cảnh giám sát từ xa, hướng tới cả kịch bản xử lý theo lô (batch) và thời gian thực (real-time), với thiết lập single-view và mở rộng multi-view.

- Sử dụng các bộ dữ liệu chuẩn CASIA-B hoặc OU-MVLP làm benchmark ban đầu nhằm chuẩn hóa quy trình huấn luyện và so sánh kết quả với các nghiên cứu liên quan.

- Nghiên cứu biểu diễn dáng đi bằng Gait Energy Image (GEI) để xây dựng mô hình baseline nhẹ, sau đó mở rộng sang biểu diễn chuỗi ảnh

silhouette kết hợp Transformer nhằm khai thác đặc trưng động theo thời gian.

- Đánh giá hiệu quả hệ thống theo các protocol chuẩn, bao gồm nhận dạng cross-view, các yếu tố gây nhiễu (thay đổi trang phục, mang túi, tốc độ di chuyển) và kịch bản open-set recognition.

1.3 Phạm vi nghiên cứu

Chương 2
Cơ sở lý thuyết

2.1 Trí tuệ nhân tạo và Học sâu

Trí tuệ nhân tạo (AI) là lĩnh vực nghiên cứu máy tính nhằm mô phỏng khả năng nhận thức và ra quyết định của con người. Trong AI, học máy (Machine Learning) là phương pháp cho phép máy tính học từ dữ liệu và tự cải thiện mà không cần lập trình trực tiếp.

2.1.1 Học sâu (Deep Learning)

Học sâu (Deep Learning) là một nhánh con của học máy, nổi bật nhờ khả năng tự động học đặc trưng từ dữ liệu. Học sâu sử dụng các mạng nơ-ron nhiều tầng để học các biểu diễn (representations) trừu tượng của dữ liệu. Mỗi tầng trong mạng nơ-ron học một mức độ trừu tượng khác nhau của đặc trưng dữ liệu. Điều này đặc biệt hữu ích trong các bài toán phức tạp như nhận diện hình ảnh, xử lý ngôn ngữ tự nhiên (NLP) và phân tích video.

Các loại mô hình trong học sâu:

- **Mạng Nơ-ron Tích chập (CNN)**: Phù hợp với dữ liệu có cấu trúc không gian như ảnh. Mô hình này học các đặc trưng hình ảnh từ đơn giản đến phức tạp thông qua các lớp tích chập (convolution).
- **Mạng Nơ-ron Recurrent (RNN)**: Thích hợp cho dữ liệu chuỗi thời gian hoặc dữ liệu tuần tự như văn bản và âm thanh.
- **Mạng Nơ-ron Generative (GAN)**: Dùng để tạo ra dữ liệu mới (ảnh, văn bản), ứng dụng trong tạo hình ảnh giả lập (deepfakes), sáng tạo nghệ thuật và phát hiện bất thường.

2.2 Mạng nơ-ron tích chập (CNN)

2.2.1 Tổng quan về CNN

Mạng nơ-ron tích chập (Convolutional Neural Network - CNN) là một trong những kiến trúc đột phá nhất trong lĩnh vực Học sâu, đóng vai trò xương sống cho hầu hết các hệ thống thị giác máy tính hiện đại. Khác với các mạng nơ-ron truyền thống (như Multi-Layer Perceptron - MLP) coi đầu vào là một vector phẳng, CNN được thiết kế đặc biệt để xử lý dữ liệu dạng lưới, tiêu biểu là ảnh kỹ thuật số (ma trận 2D hoặc 3D).

Cảm hứng xây dựng CNN xuất phát từ các nghiên cứu về vỏ não thị giác của động vật (Visual Cortex) do Hubel và Wiesel thực hiện. Các nhà khoa học phát hiện ra rằng các nơ-ron trong não không phản ứng với toàn bộ khung cảnh cùng lúc, mà mỗi nơ-ron chỉ phản ứng với một vùng nhỏ cục bộ (gọi là *Receptive Field*). Tập hợp các vùng cục bộ này ghép lại sẽ tạo nên nhận thức về toàn bộ hình ảnh. CNN mô phỏng cơ chế này thông qua phép toán tích chập (convolution).

2.2.2 Tại sao CNN vượt trội trong xử lý ảnh?

Trước khi CNN trở nên phổ biến, các mạng nơ-ron kết nối đầy đủ (Fully Connected Networks) gặp rất nhiều khó khăn khi xử lý ảnh vì hai lý do chính:

- **Bùng nổ tham số (Parameter Explosion)**: Một bức ảnh màu nhỏ kích thước $224 \times 224 \times 3$ có tới 150,528 điểm dữ liệu. Nếu kết nối vào một lớp ẩn có 1000 nơ-ron, số lượng trọng số cần học sẽ lên tới hơn 150 triệu. Điều này dẫn đến chi phí tính toán khổng lồ và hiện tượng quá khớp (overfitting) nghiêm trọng.

- **Mất thông tin không gian**: Việc duỗi thẳng (flatten) ảnh thành vector khiến mạng mất đi thông tin về cấu trúc không gian 2D (ví dụ: mối liên hệ giữa các pixel lân cận tạo nên các đường nét).

CNN giải quyết triệt để các vấn đề trên nhờ hai cơ chế cốt lõi:

1. **Kết nối cục bộ (Local Connectivity)**: Mỗi nơ-ron trong lớp tích chập chỉ kết nối với một vùng nhỏ của ảnh đầu vào thay vì toàn bộ ảnh. Điều này giúp mạng tập trung vào các đặc trưng cục bộ quan trọng.

2. **Chia sẻ trọng số (Parameter Sharing)**: Cùng một bộ lọc (filter/kernel) được sử dụng quét qua toàn bộ bức ảnh. Giả định rằng một đặc trưng (ví dụ: cạnh dọc) nếu có ích ở góc trái ảnh thì cũng có ích ở góc phải. Cơ chế này giúp giảm đáng kể số lượng tham số cần huấn luyện.

2.2.3 Cơ chế trích xuất đặc trưng phân tầng

Sức mạnh thực sự của CNN nằm ở khả năng học phân tầng (Hierarchical Learning). Khi đi qua các lớp của mạng, mức độ trừu tượng của thông tin tăng dần:

- **Các tầng đầu (Low-level features)**: Học các đặc trưng cơ bản như đường thẳng, cạnh, góc, màu sắc.
- **Các tầng giữa (Mid-level features)**: Kết hợp các cạnh để tạo thành các hình dạng phức tạp hơn như mắt, mũi, miệng, kết cấu da.
- **Các tầng cuối (High-level features)**: Tổng hợp thành các đối tượng hoàn chỉnh (khuôn mặt người, đầu hói, tóc).

Quá trình này diễn ra hoàn toàn tự động thông qua huấn luyện (backpropagation) mà không cần con người thiết kế đặc trưng thủ công như các phương pháp máy học cổ điển (SIFT, HOG).

2.2.4 Các thành phần của CNN

CNN bao gồm ba loại lớp chính:

1. Lớp tích chập (Convolutional Layer)

Đây là thành phần cốt lõi của mạng CNN, chịu trách nhiệm trích xuất các đặc trưng từ dữ liệu đầu vào. Khác với lớp kết nối đầy đủ (Fully Connected) xử

lý toàn bộ dữ liệu cùng lúc, lớp tích chập sử dụng cơ chế "cửa sổ trượt"(sliding window) để quét qua ảnh.

Cơ chế hoạt động: Một tập hợp các bộ lọc (filters/kernels) có kích thước nhỏ (thường là 3×3 hoặc 5×5) sẽ trượt qua toàn bộ chiều rộng và chiều cao của ảnh đầu vào. Tại mỗi vị trí, bộ lọc thực hiện phép nhân từng phần tử (element-wise multiplication) với vùng ảnh tương ứng và tính tổng các kết quả đó để tạo ra một giá trị duy nhất trên bản đồ đặc trưng (Feature Map).

Mô hình toán học: Giả sử đầu vào là ma trận X và bộ lọc là ma trận W có kích thước $K \times K$. Giá trị tại vị trí (i, j) của bản đồ đặc trưng đầu ra Y được tính theo công thức:

$$Y_{i,j} = \left(\sum_{m=0}^{K-1} \sum_{n=0}^{K-1} X_{i+m,j+n} \cdot W_{m,n} \right) + b \tag{2.1}$$

Trong đó:

- $X_{i+m,j+n}$: Giá trị pixel tại vị trí tương ứng trên ảnh đầu vào.
- $W_{m,n}$: Trọng số tại vị trí (m,n) của bộ lọc (tham số cần học).
- b: Hệ số bias (độ lệch), giúp điều chỉnh ngưỡng kích hoạt.

Các siêu tham số quan trọng: Hiệu suất và kích thước đầu ra của lớp tích chập phụ thuộc vào ba yếu tố chính:

- **Kích thước bộ lọc (Kernel Size K)**: Quyết định vùng cục bộ mà mạng "nhìn"thấy (Receptive Field). Kích thước nhỏ (3×3) giúp bắt các chi tiết tinh, trong khi kích thước lớn bắt các đặc trưng tổng quát hơn.
- **Bước trượt (Stride S)**: Là khoảng cách di chuyển của bộ lọc sau mỗi bước tính toán. $S = 1$ giữ nguyên độ phân giải không gian, trong khi $S > 1$ làm giảm kích thước ảnh đầu ra (downsampling).
- **Phần đệm (Padding P)**: Là việc thêm các giá trị 0 bao quanh viền ảnh đầu vào. Kỹ thuật này giúp giữ nguyên kích thước ảnh sau khi tích chập và cho phép mạng khai thác thông tin ở vùng biên ảnh.

Công thức tính kích thước đầu ra: Nếu ảnh đầu vào có kích thước $W_{in} \times H_{in}$, kích thước ảnh đầu ra $W_{out} \times H_{out}$ được tính như sau:

$$W_{out} = \left\lfloor \frac{W_{in} - K + 2P}{S} \right\rfloor + 1 \tag{2.2}$$

Công thức này giải thích lý do tại sao trong thiết kế mô hình của đồ án, việc sử dụng $K = 3, P = 1, S = 1$ giúp giữ nguyên kích thước ảnh 224×224 qua các lớp tích chập.

2. Lớp kích hoạt phi tuyến ReLU (Rectified Linear Unit)

Nếu các lớp tích chập đóng vai trò là bộ trích xuất đặc trưng tuyến tính, thì lớp kích hoạt (Activation Layer) đóng vai trò quyết định trong việc giúp mạng nơ-ron học được các mối quan hệ phức tạp và phi tuyến tính của dữ liệu. Trong đồ án này, hàm kích hoạt **ReLU** được lựa chọn sử dụng sau mỗi lớp tích chập.

Định nghĩa toán học: ReLU (Rectified Linear Unit) là một hàm toán học đơn giản thực hiện ngưỡng hóa các giá trị tại điểm 0. Công thức được định nghĩa như sau:

$$f(x) = \max(0, x) = \begin{cases} x & \text{nếu } x > 0 \\ 0 & \text{nếu } x \leq 0 \end{cases} \tag{2.3}$$

Về mặt trực quan, hàm này giữ nguyên các giá trị dương (đặc trưng quan trọng) và triệt tiêu các giá trị âm (nhiễu hoặc đặc trưng không liên quan) về 0.

Tại sao ReLU vượt trội hơn Sigmoid/Tanh? Trước khi ReLU phổ biến, các hàm như Sigmoid hay Tanh thường được sử dụng. Tuy nhiên, ReLU đã trở thành chuẩn mực cho CNN hiện đại nhờ ba ưu điểm vượt trội:

- **Khắc phục vấn đề biến mất đạo hàm (Vanishing Gradient Problem):** Khi huấn luyện các mạng sâu, đạo hàm của hàm Sigmoid rất nhỏ (tối đa là 0.25) và tiến dần về 0 ở hai đầu, khiến trọng số ở các lớp đầu tiên gần như không được cập nhật. Ngược lại, đạo hàm của ReLU là 1 (với $x > 0$)

hoặc 0. Điều này giúp gradient lan truyền ngược (backpropagation) qua nhiều tầng mà không bị suy giảm, cho phép huấn luyện các mạng CNN rất sâu.

- **Tính thưa thớt (Sparsity):** Trong một mạng nơ-ron dày đặc, việc tất cả các nơ-ron đều kích hoạt cùng lúc sẽ gây tốn kém tài nguyên và dễ bị nhiễu. ReLU ép một phần lớn các nơ-ron có giá trị âm về 0 (trạng thái "bất hoạt"). Sự thưa thớt này giúp mô hình gọn nhẹ hơn và tập trung vào các đặc trưng thực sự nổi bật (ví dụ: chỉ kích hoạt khi thấy đường nét rõ ràng).

- **Hiệu quả tính toán (Computational Efficiency):** Việc tính toán hàm mũ e^x (trong Sigmoid) tốn nhiều tài nguyên phần cứng. ReLU chỉ cần một phép so sánh đơn giản $\max(0, x)$, giúp tăng tốc độ huấn luyện và suy diễn đáng kể trên GPU.

3. Lớp gộp (Pooling Layer)

Sau các lớp tích chập và hàm kích hoạt, kích thước không gian của bản đồ đặc trưng (Feature Map) thường vẫn còn lớn và chứa nhiều thông tin dư thừa. Lớp gộp (Pooling Layer), hay còn gọi là lớp lấy mẫu xuống (Downsampling), được sử dụng để giải quyết vấn đề này.

Vai trò và Chức năng: Lớp gộp có hai nhiệm vụ chiến lược trong kiến trúc CNN:

1. **Giảm chiều dữ liệu (Dimensionality Reduction):** Giảm kích thước không gian (chiều rộng W và chiều cao H) của ảnh, từ đó giảm đáng kể số lượng tham số và khối lượng tính toán cho các lớp phía sau.

2. **Tăng tính bất biến (Invariance):** Giúp mô hình trở nên mạnh mẽ hơn trước các biến đổi nhỏ về vị trí của đối tượng trong ảnh. Ví dụ: dù khuôn mặt dịch chuyển nhẹ sang trái hay phải, kết quả của lớp pooling vẫn không thay đổi nhiều.

Các kỹ thuật Pooling phổ biến: Có hai phương pháp gộp chính thường được sử dụng:

- **Max Pooling (Gộp cực đại):** Chỉ lấy giá trị lớn nhất trong vùng cửa sổ trượt. Đây là kỹ thuật được sử dụng trong đồ án này vì nó hoạt động như một bộ lọc "chọn lọc đặc trưng", chỉ giữ lại những đặc trưng nổi bật nhất (ví dụ: biên cạnh rõ nét nhất) và loại bỏ các vùng nền nhiễu.

- **Average Pooling (Gộp trung bình):** Tính giá trị trung bình của các phần tử trong cửa sổ. Phương pháp này thường làm mịn ảnh và ít được sử dụng hơn trong các bài toán nhận diện đặc trưng sắc nét.

Mô hình toán học (Max Pooling): Với một cửa sổ trượt kích thước $K \times K$ và bước nhảy S, giá trị đầu ra tại vị trí (i, j) được tính bằng:

$$Y_{i,j} = \max_{m,n \in [0, K-1]} \left(X_{i \times S + m,\ j \times S + n} \right) \tag{2.4}$$

Trong mô hình đề xuất, chúng tôi sử dụng Max Pooling với kích thước 2×2 và bước nhảy $S = 2$. Cấu hình này giúp giảm kích thước ảnh đi một nửa (ví dụ: từ 224×224 xuống 112×112) nhưng vẫn giữ lại được 75% thông tin quan trọng nhất (theo nguyên lý giữ lại giá trị kích hoạt mạnh nhất).

4. Lớp kết nối đầy đủ (Fully Connected Layer - FC)

Nếu các lớp tích chập và gộp đóng vai trò là bộ trích xuất đặc trưng (Feature Extractor) để "nhìn"và tóm tắt các chi tiết trong ảnh, thì lớp kết nối đầy đủ (Fully Connected Layer) đóng vai trò là bộ phân loại (Classifier) để "ra quyết định"dựa trên các đặc trưng đó.

Quy trình hoạt động: Quá trình xử lý tại lớp này diễn ra qua hai bước chính:

1. **Duỗi phẳng (Flattening):** Đầu ra của lớp tích chập cuối cùng là một khối 3D (ví dụ: $64 \times 7 \times 7$). Trước khi đưa vào lớp FC, khối này phải được "duỗi"thành một vector 1D dài (ví dụ: vector có kích thước 3136). Quá trình này chuyển đổi dữ liệu từ không gian đặc trưng hình ảnh sang dạng vector đặc trưng đại số.

2. **Kết nối toàn vẹn:** Giống như mạng nơ-ron truyền thống (MLP), mỗi nơ-ron trong lớp FC được kết nối với tất cả các nơ-ron của lớp trước đó.

Tại đây, mô hình thực hiện phép tính tổng trọng số để tổng hợp thông tin cục bộ thành thông tin toàn cục.

Mô hình toán học: Đầu ra y của một nơ-ron trong lớp FC được tính theo công thức:

$$y = f\left(\sum_{i=1}^{N} w_i x_i + b\right) \tag{2.5}$$

Trong đó x là vector đầu vào (đã được duỗi), w là trọng số, b là bias và f là hàm kích hoạt.

Vai trò trong bài toán nhận diện: Trong ngữ cảnh của bài toán nhận diện đầu hói, các nơ-ron trong lớp FC sẽ học cách kết hợp các đặc trưng bậc cao (như "đường chân tóc cao", "vùng trán láng mịn") để đưa ra điểm số (logits) cho từng lớp **Bald** hoặc **NotBald**. Tuy nhiên, do số lượng tham số tại lớp này thường rất lớn (hàng triệu tham số), lớp FC rất dễ gây ra hiện tượng quá khớp (overfitting), do đó kỹ thuật *Dropout* thường được áp dụng kèm theo tại tầng này.

2.2.5 Vì sao CNN lại hiệu quả trong nhận diện ảnh?

CNN hoạt động hiệu quả trong nhận diện ảnh nhờ vào ba đặc điểm sau:
- **Tính chất không gian**: Các bộ lọc trong CNN học được đặc trưng không gian của ảnh, giúp nhận diện các đối tượng bất kể chúng ở đâu trong ảnh.
- **Tính chia sẻ bộ lọc**: Bộ lọc được sử dụng cho toàn bộ ảnh, giúp mô hình học được đặc trưng chung mà không phải học lại từ đầu ở mỗi vị trí.
- **Cấu trúc phân tầng**: Mô hình CNN có thể học các đặc trưng từ cơ bản đến phức tạp, từ đó giúp phân loại các đối tượng chính xác hơn.

2.3 Hàm mất mát Cross-Entropy

Trong các bài toán phân loại sử dụng mạng nơ-ron, hàm mất mát (Loss Function) đóng vai trò như một "chiếc la bàn", định hướng cho việc điều chỉnh trọng số thông qua lan truyền ngược. Đối với bài toán phân loại ảnh, **Cross-Entropy Loss** (Mất mát chéo) là sự lựa chọn tiêu chuẩn nhờ khả năng trừng phạt mạnh các dự đoán sai lệch.

2.3.1 Cơ sở lý thuyết

Cross-Entropy có nguồn gốc từ Lý thuyết thông tin (Information Theory) của Claude Shannon. Nó đo lường sự khác biệt giữa hai phân phối xác suất: phân phối xác suất thực tế y (Ground Truth) và phân phối xác suất dự đoán $\hat{y}$ (Prediction) từ mô hình.

Mục tiêu của quá trình huấn luyện là cực tiểu hóa giá trị này, tức là kéo phân phối dự đoán $\hat{y}$ lại gần nhất có thể với phân phối thực y.

2.3.2 Công thức toán học

Công thức tổng quát của Cross-Entropy cho một mẫu dữ liệu với C lớp phân loại được định nghĩa như sau:

$$\mathscr{L}_{CE} = -\sum_{i=1}^{C} y_i \log(\hat{y}_i) \tag{2.6}$$

Trong đó:

- C: Tổng số lớp (trong bài toán này $C = 2$).
- y_i: Nhãn thực tế của lớp i (dưới dạng one-hot encoding). Ví dụ: ảnh Bald có nhãn $y = [1, 0]$.
- $\hat{y}_i$: Xác suất dự đoán mô hình dành cho lớp i (đầu ra của hàm Softmax).
- log: Logarithm tự nhiên (cơ số e).

2.3.3 Trường hợp phân loại nhị phân (Binary Cross-Entropy)

Vì đề tài tập trung vào việc nhận diện hai trạng thái đối lập (**Bald** và **NotBald**), công thức trên có thể được viết gọn lại thành Binary Cross-Entropy. Đây là hàm mục tiêu trực tiếp được tối ưu hóa trong đồ án:

$$\mathscr{L} = -\left[y \log(\hat{y}) + (1-y)\log(1-\hat{y})\right] \tag{2.7}$$

Tại sao hàm này hiệu quả? Hàm Logarithm có đặc tính tiến tới $-\infty$ khi giá trị tiến về 0. Do đó:

- Nếu nhãn thực là 1 ($y = 1$) nhưng mô hình dự đoán sai là 0 ($\hat{y} \approx 0$), thành phần $\log(\hat{y})$ sẽ tiến tới $-\infty$, làm giá trị Loss $\mathscr{L}$ tăng vọt lên rất cao.

- Điều này tạo ra một gradient rất lớn, buộc mô hình phải điều chỉnh trọng số mạnh mẽ để sửa chữa sai lầm này ngay lập tức. Đây là ưu điểm vượt trội của Cross-Entropy so với hàm lỗi bình phương trung bình (MSE) trong bài toán phân loại.

2.4 Thuật toán tối ưu Adam

Trong quá trình huấn luyện mạng nơ-ron, việc lựa chọn thuật toán tối ưu (Optimizer) quyết định tốc độ hội tụ và hiệu suất cuối cùng của mô hình. Đồ án sử dụng thuật toán **Adam (Adaptive Moment Estimation)**, một phương pháp tối ưu hóa ngẫu nhiên được Kingma và Ba giới thiệu năm 2014. Adam được xem là sự kết hợp ưu điểm của hai thuật toán phổ biến trước đó:

- **Momentum:** Giúp tăng tốc độ di chuyển theo hướng có gradient cùng dấu liên tục và giảm dao động phương sai.

- **RMSProp:** Tự động điều chỉnh tốc độ học (adaptive learning rate) cho từng tham số dựa trên độ lớn của gradient gần đây.

2.4.1 Cơ chế hoạt động

Adam duy trì hai ước lượng trung bình động của gradient (moment bậc 1) và bình phương gradient (moment bậc 2). Quy trình cập nhật tham số tại bước thời gian t diễn ra như sau:

1. Tính toán Gradient:

$$g_t = \nabla_\theta J(\theta_{t-1})$$

2. Cập nhật các moment (có trọng số):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \qquad \text{(Ước lượng moment bậc 1)} \tag{2.8}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \qquad \text{(Ước lượng moment bậc 2)} \tag{2.9}$$

3. Hiệu chỉnh chệch (Bias Correction): Do các vector m_t và v_t được khởi tạo bằng 0, chúng có xu hướng bị lệch về 0 trong các bước đầu tiên (đặc biệt khi β_1, β_2 gần bằng 1). Bước hiệu chỉnh này giúp khử bỏ độ lệch đó:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \tag{2.10}$$

4. Cập nhật tham số:

$$\theta_t = \theta_{t-1} - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \tag{2.11}$$

2.4.2 Lý do lựa chọn Adam

Trong bài toán nhận diện đầu hói với dữ liệu ảnh khuôn mặt, Adam được lựa chọn nhờ các ưu điểm vượt trội:

- **Hội tụ nhanh:** Nhờ cơ chế Momentum, Adam thường đạt được loss thấp nhanh hơn so với SGD truyền thống.
- **Không cần tinh chỉnh Learning Rate quá nhiều:** Adam có khả năng tự thích nghi, hoạt động tốt với thiết lập mặc định ($\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \varepsilon = 10^{-8}$).
- **Hiệu quả với dữ liệu thưa và nhiễu:** Phù hợp với đặc thù của các bài toán thị giác máy tính sử dụng CNN.

2.5 Hàm kích hoạt Softmax

Trong kiến trúc mạng nơ-ron phân loại, lớp cuối cùng (Output Layer) thường trả về một vector các giá trị thực gọi là *logits*. Các giá trị này có thể âm, dương hoặc rất lớn, không mang ý nghĩa trực quan về mặt xác suất. Hàm

Softmax được sử dụng để giải quyết vấn đề này bằng cách chuẩn hóa vector logits thành một phân phối xác suất hợp lệ.

2.5.1 Định nghĩa toán học

Cho một vector đầu vào $z = (z_1, z_2, \ldots, z_K)$ gồm K phần tử (tương ứng với K lớp). Hàm Softmax, $\sigma(z)_i$, xác định xác suất cho lớp thứ i như sau:

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \quad \text{với } i = 1, \ldots, K \tag{2.12}$$

Trong đó:

- z_i: Giá trị logit thô của lớp i (đầu ra của lớp Fully Connected cuối cùng).
- e^{z_i}: Hàm mũ cơ số tự nhiên. Việc sử dụng hàm mũ giúp đảm bảo kết quả luôn dương và nới rộng khoảng cách giữa các giá trị (làm nổi bật lớp có điểm số cao nhất).
- $\sum e^{z_j}$: Tổng các giá trị mũ của tất cả các lớp, đóng vai trò là hệ số chuẩn hóa.

2.5.2 Tính chất quan trọng

Hàm Softmax có hai tính chất cốt lõi khiến nó trở thành tiêu chuẩn cho các bài toán phân loại:

1. **Phạm vi giá trị:** Đầu ra của Softmax luôn nằm trong khoảng $(0, 1)$. Điều này cho phép diễn giải kết quả đầu ra là "xác suất"hoặc "độ tin cậy"của mô hình đối với từng lớp.
2. **Tổng bằng 1:** Tổng xác suất của tất cả các lớp luôn bằng 1:

$$\sum_{i=1}^{K} \sigma(z)_i = 1$$

   Tính chất này đảm bảo tính nhất quán của phân phối xác suất (ví dụ: mô hình dự báo 70% là Hói thì bắt buộc 30% còn lại là Không Hói).

2.5.3 Ứng dụng trong đồ án

Trong hệ thống nhận diện đầu hói (phân loại 2 lớp), mô hình trả về vector 2 chiều (z_1, z_2). Hàm Softmax sẽ chuyển đổi vector này thành (p_1, p_2) tương ứng với xác suất của lớp **NotBald** và **Bald**. Nhãn dự đoán cuối cùng $\hat{y}$ được xác định bằng cách chọn lớp có xác suất cao nhất:

$$\hat{y} = \text{argmax}(p_1, p_2)$$

Chương 3

Dataset và tiền xử lý dữ liệu

3.1 Dataset sử dụng

- CASIA-B: Bộ dữ liệu nhận diện dáng đi chuẩn, gồm nhiều đối tượng với đa góc nhìn (multi-view) và các điều kiện khác nhau như đi bình thường, thay đổi trang phục và mang vật dụng.
- OU-MVLP: Bộ dữ liệu dáng đi quy mô lớn với hơn 10.000 đối tượng, nhiều góc quan sát, phù hợp cho đánh giá khả năng cross-view và hiệu năng ở quy mô dân số lớn.

3.2 Tiền xử lý dữ liệu

Mục tiêu: Quá trình tiền xử lý nhằm chuẩn hóa dữ liệu đầu vào, đảm bảo tính đồng nhất về kích thước và biểu diễn trước khi đưa vào mô hình CNN để trích xuất đặc trưng dáng đi.

Đầu vào: Video dáng đi của người dùng, được thu thập từ hệ thống giám sát hoặc từ các bộ dữ liệu chuẩn dùng cho huấn luyện hoặc đánh giá.

Quy trình xử lý: Tách silhouette của người đi bộ khỏi nền; từ chuỗi silhouette tạo Gait Energy Image (GEI) hoặc giữ nguyên chuỗi ảnh theo thời gian để khai thác đặc trưng động.

Hướng mở rộng: cải tiến biểu diễn GEI để hạn chế mất mát thông tin theo thời gian, kết hợp các mô hình học sâu xử lý chuỗi như 3D-CNN hoặc Transformer nhằm nâng cao hiệu quả nhận diện.

Chương 4

Xây dựng mô hình

4.1 Kiến trúc mô hình

4.1.1 Ý tưởng thiết kế

Mô hình sử dụng 3 khối tích chập để học đặc trưng từ thô đến tinh, sau đó flatten và fully-connected để phân loại 2 lớp. Dropout ở fully-connected giúp giảm overfitting.

4.1.2 Đoạn code định nghĩa CNN

Dưới đây là đoạn mã định nghĩa mô hình CNN:

img2.png

Hình 4.1: Code định nghĩa mô hình CNN

4.1.3 Kích thước tensor qua các tầng

Bảng 4.1: Kích thước đầu ra theo tầng (input: $3\times224\times224$)

Tầng	Cấu hình	Output shape
Conv1 + ReLU	$3\rightarrow16$, k=3, pad=1	$16\times224\times224$
MaxPool1	2×2	$16\times112\times112$
Conv2 + ReLU	$16\rightarrow32$, k=3, pad=1	$32\times112\times112$
MaxPool2	2×2	$32\times56\times56$
Conv3 + ReLU	$32\rightarrow64$, k=3, pad=1	$64\times56\times56$
MaxPool3	2×2	$64\times28\times28$
Flatten	–	64*28*28 = 50176
FC1 + ReLU + Dropout	$50176\rightarrow256$	256
FC2 (logits)	$256\rightarrow2$	2

4.2 Huấn luyện mô hình

4.2.1 Thiết lập huấn luyện (Loss, Optimizer, Epoch)

Trước khi tiến hành huấn luyện, chúng tôi thiết lập môi trường tính toán và các thành phần cốt lõi của quá trình học như sau:

- **Thiết bị tính toán (Device):** Hệ thống tự động kiểm tra sự sẵn có của GPU (CUDA). Việc sử dụng GPU giúp tăng tốc độ tính toán các phép nhân ma trận trong CNN lên gấp nhiều lần so với CPU truyền thống.
- **Hàm mất mát (Loss Function):** Sử dụng `CrossEntropyLoss`. Đây là hàm mục tiêu tiêu chuẩn cho bài toán phân loại, giúp đo lường độ sai lệch giữa phân phối xác suất dự đoán và nhãn thực tế, đồng thời phạt nặng các dự đoán sai có độ tin cậy cao.
- **Thuật toán tối ưu (Optimizer):** Sử dụng `Adam` với tốc độ học (learning rate) khởi tạo là $\alpha = 0.008$. Adam được lựa chọn nhờ khả năng tự điều

chỉnh learning rate cho từng tham số, giúp mô hình hội tụ nhanh và ổn định hơn so với SGD thuần túy.

- **Chu kỳ huấn luyện (Epochs):** Tổng số epoch được thiết lập là 25. Đây là số lần toàn bộ tập dữ liệu được đưa qua mạng nơ-ron để cập nhật trọng số.

Dưới đây là đoạn mã thiết lập huấn luyện:

```
img3.png
```

Hình 4.2: Thiết lập huấn luyện

4.2.2 Thuật toán huấn luyện và validation

Luồng thuật toán theo epoch:

1. **Train**: `model.train()`, lặp theo batch, cập nhật trọng số bằng Adam.

2. **Validation**: `model.eval()` + `torch.no_grad()`, tính accuracy.

4.2.3 Quy trình huấn luyện và đánh giá (Training & Validation Loop)

Quá trình huấn luyện mô hình được thực hiện qua một vòng lặp lớn theo số lượng *epoch* đã thiết lập. Trong mỗi epoch, dữ liệu được xử lý qua hai giai đoạn riêng biệt: giai đoạn huấn luyện (Training Phase) và giai đoạn đánh giá (Validation Phase).

Giai đoạn huấn luyện (Training Phase)

Trong giai đoạn này, mô hình được đặt ở chế độ `model.train()`. Điều này rất quan trọng để kích hoạt các lớp như *Dropout* hoặc *Batch Normalization* hoạt động đúng chức năng. Các bước thực hiện bao gồm:

1. **Xóa Gradient (Zero Grad):** Gọi lệnh `optimizer.zero_grad()` đầu mỗi batch để xóa các đạo hàm cũ. Trong PyTorch, gradient được cộng dồn mặc định, nên bước này là bắt buộc để tránh tính toán sai hướng cập nhật.

2. **Lan truyền xuôi (Forward Pass):** Dữ liệu ảnh đầu vào đi qua mạng CNN để tạo ra vector dự đoán. Hàm mất mát (Loss) được tính toán dựa trên sự sai lệch giữa dự đoán này và nhãn thực tế.

3. **Lan truyền ngược (Backward Pass):** Gọi lệnh `loss.backward()` để tính đạo hàm (gradient) của hàm mất mát theo từng trọng số trong mạng.

4. **Cập nhật trọng số (Optimizer Step):** Thuật toán Adam sử dụng gradient vừa tính được để cập nhật trọng số của mô hình thông qua lệnh `optimizer.step()`, giúp giảm thiểu giá trị Loss.

Giai đoạn đánh giá (Validation Phase)

Sau khi huấn luyện xong một epoch, mô hình chuyển sang chế độ `model.eval()` để đánh giá hiệu năng trên tập dữ liệu kiểm thử (Validation Set).

- **Tắt tính toán Gradient:** Sử dụng ngữ cảnh `torch.no_grad()` để vô hiệu hóa việc tính toán đạo hàm. Việc này giúp tiết kiệm bộ nhớ GPU và tăng tốc độ xử lý vì ta không cần cập nhật trọng số trong giai đoạn này.
- **Tính toán độ chính xác:** Kết quả dự đoán (logits) được đưa qua hàm argmax để chọn ra nhãn có xác suất cao nhất. Độ chính xác (Accuracy) được tính bằng tỷ lệ số mẫu dự đoán đúng trên tổng số mẫu kiểm thử.

Dưới đây là đoạn mã hiện thực hóa quy trình trên:

img4.png

Hình 4.3: Mã nguồn vòng lặp huấn luyện và đánh giá mô hình

Kết thúc mỗi epoch, hệ thống sẽ in ra các thông số: *Train Loss* (để theo

dõi độ hội tụ), *Val Loss* (để phát hiện overfitting) và *Val Accuracy* (để đánh giá hiệu năng thực tế).

Chương 5

Đánh giá kết quả

5.1 Độ chính xác

5.1.1 Accuracy tổng thể

$$Accuracy = \frac{\text{\#dự đoán đúng}}{\text{\#mẫu}} \times 100\%$$

5.1.2 Accuracy theo lớp (quan trọng khi lệch lớp)

$$Acc_c = \frac{\text{\#dự đoán đúng lớp } c}{\text{\#mẫu lớp } c} \times 100\%$$

5.1.3 Đoạn code đánh giá

img5.png

Hình 5.1: Đánh giá accuracy tổng thể và theo lớp

5.1.4 Độ chính xác của mô hình

Screenshot 2026-01-07 224403.png

Hình 5.2: Độ chính xác của mô hình

5.2 Nhận xét và đánh giá chi tiết

Dựa trên kết quả thực nghiệm và quá trình huấn luyện, nhóm thực hiện đưa ra những nhận xét chi tiết về ưu điểm cũng như các tồn tại của hệ thống:

5.2.1 Về chiến lược xử lý dữ liệu

- **Hiệu quả của Augmentation:** Việc áp dụng các kỹ thuật tăng cường dữ liệu như lật ngang (Horizontal Flip) và xoay nhẹ (Rotation) đã giúp mô hình học được các đặc trưng bất biến của khuôn mặt. Điều này đặc

biệt quan trọng vì trong thực tế, người dùng có thể chụp ảnh ở các góc nghiêng khác nhau. Tuy nhiên, các kỹ thuật hiện tại chưa bao phủ được các biến thể phức tạp hơn như thay đổi độ sáng (Brightness jitter) hay che khuất (Occlusion), dẫn đến khả năng mô hình có thể bị giảm độ chính xác trong môi trường thiếu sáng.

- **Vấn đề mất cân bằng dữ liệu:** Kết quả thực nghiệm cho thấy Accuracy tổng thể khá cao, nhưng Accuracy của lớp **Bald** thường thấp hơn lớp **NotBald**. Điều này phản ánh sự mất cân bằng trong tập dữ liệu CelebA (số lượng ảnh không hói chiếm đa số). Mô hình có xu hướng "học vẹt"theo lớp đa số để tối ưu hóa hàm loss chung, dẫn đến nguy cơ bỏ sót các trường hợp hói đầu thực sự (False Negative).

5.2.2 Về kiến trúc mô hình CNN

- **Bùng nổ tham số tại lớp Fully Connected (FC):** Kiến trúc hiện tại giữ nguyên kích thước không gian qua các lớp Conv (nhờ padding=1) và chỉ giảm kích thước qua 3 lớp Pooling ($224 \rightarrow 112 \rightarrow 56 \rightarrow 28$). Khi duỗi phẳng (Flatten), ta thu được vector kích thước rất lớn: $64 \times 28 \times 28 = 50,176$. Việc kết nối vector này vào lớp ẩn 256 nơ-ron tạo ra số lượng trọng số khổng lồ: $50,176 \times 256 \approx 12.8$ triệu tham số chỉ riêng tại lớp này.

- **Nguy cơ Overfitting:** Số lượng tham số quá lớn so với lượng dữ liệu huấn luyện khiến mô hình dễ rơi vào trạng thái Overfitting (học thuộc lòng dữ liệu train thay vì học đặc trưng). Mặc dù đã sử dụng lớp *Dropout* để giảm thiểu tác động này, nhưng về lâu dài, việc thay thế lớp FC bằng kỹ thuật *Global Average Pooling (GAP)* sẽ là giải pháp tối ưu hơn để giảm chiều dữ liệu.

5.2.3 Về khả năng ứng dụng thực tế

- Mô hình hoạt động tốt với các ảnh chân dung tiêu chuẩn (nền đơn giản, ánh sáng tốt).

- Đối với các trường hợp "biên"(edge cases) như người đội mũ, tóc quá sáng màu trùng với nền, hoặc hói chữ M (hói nhẹ), mô hình vẫn còn gặp khó khăn trong việc phân loại chính xác. Điều này cho thấy mạng CNN hiện tại mới chỉ học được các đặc trưng cục bộ (texture da đầu) mà chưa thực sự hiểu được cấu trúc toàn cục của khuôn mặt.

Chương 6

Xây dựng giao diện Web

6.1 Công nghệ sử dụng

Giao diện web được xây dựng sử dụng **Streamlit**, một thư viện Python phổ biến cho phép tạo ra ứng dụng web tương tác đơn giản và nhanh chóng. Streamlit đặc biệt hữu ích trong việc triển khai các mô hình học máy, cung cấp khả năng tạo giao diện người dùng để tương tác trực tiếp với mô hình học máy mà không cần phải viết mã front-end phức tạp.

- **Streamlit**: Thư viện Python giúp xây dựng giao diện web cho ứng dụng học máy. Nó cho phép người dùng tải ảnh lên, xử lý ảnh và nhận kết quả phân loại ngay lập tức.
- **PIL (Python Imaging Library)**: Được sử dụng để mở và xử lý ảnh trước khi đưa vào mô hình.
- **PyTorch**: Thư viện học sâu để định nghĩa và huấn luyện mô hình CNN.
- **Torchvision**: Dùng để áp dụng các phép biến đổi tiền xử lý trên ảnh đầu vào.

6.2 Luồng hoạt động

Ứng dụng Streamlit hoạt động theo một quy trình đơn giản và dễ hiểu:

1. **Tải ảnh**: Người dùng tải lên một ảnh khuôn mặt qua giao diện web. Ảnh này sẽ được sử dụng làm đầu vào cho mô hình phân loại.

2. **Tiền xử lý ảnh**: Ảnh được tiền xử lý (resize và normalize) để phù hợp với mô hình học sâu. Các phép biến đổi này giúp ảnh có kích thước cố định và dễ dàng đưa vào mô hình.

3. **Dự đoán với mô hình CNN**: Sau khi tiền xử lý xong, ảnh sẽ được đưa vào mô hình CNN đã huấn luyện để phân loại thành 2 lớp: **Bald** hoặc **Not Bald**.

4. **Hiển thị kết quả**: Kết quả phân loại sẽ được hiển thị trên giao diện web, cho phép người dùng xem kết quả dự đoán ngay lập tức.

6.3 Mã nguồn của giao diện web

Đoạn mã dưới đây sử dụng **Streamlit** để xây dựng giao diện người dùng. Mô hình CNN được tải lên và sử dụng để dự đoán ảnh đầu hói từ ảnh khuôn mặt.

Screenshot 2026-01-07 225057.png

Hình 6.1: Giao diện web Streamlit

6.4 Đánh giá và Nhận xét

Việc tích hợp mô hình Deep Learning vào giao diện web mang lại cái nhìn trực quan về tính ứng dụng của đề tài. Nhóm thực hiện đưa ra các đánh giá chi tiết sau:

6.4.1 Về trải nghiệm người dùng (UX)

- **Tính trực quan và dễ sử dụng:** Giao diện được thiết kế tối giản, tập trung vào chức năng cốt lõi là tải ảnh và xem kết quả. Luồng thao tác (Workflow) chỉ gồm 2 bước (Upload $\rightarrow$ Dự đoán), giúp người dùng không cần kiến thức kỹ thuật vẫn có thể tương tác với hệ thống.

- **Phản hồi thời gian thực:** Nhờ việc tối ưu hóa quy trình tiền xử lý ảnh, độ trễ từ lúc tải ảnh lên đến khi hiện kết quả là rất thấp (dưới 1 giây trên GPU), mang lại trải nghiệm mượt mà.

6.4.2 Về khía cạnh kỹ thuật

- **Che giấu sự phức tạp:** Ứng dụng đã đóng gói thành công toàn bộ quy trình phức tạp bên dưới (Load model, Transform ảnh, Forward pass qua CNN, Softmax) thành một giao diện nút bấm đơn giản. Đây là bước chuyển đổi quan trọng từ "mã lệnh nghiên cứu"sang "sản phẩm ứng dụng".

- **Khả năng triển khai nhanh (Rapid Prototyping):** Sử dụng Streamlit giúp giảm thiểu thời gian viết mã Front-end (HTML/CSS/JS), cho phép nhóm tập trung tối đa thời gian vào việc tối ưu thuật toán và mô hình (Backend) - ưu tiên hàng đầu của đồ án Học máy.

6.4.3 Hạn chế cần khắc phục

Bên cạnh các ưu điểm, ứng dụng web hiện tại vẫn còn một số hạn chế:

- **Xử lý đơn nhiệm:** Hệ thống hiện tại chỉ hỗ trợ xử lý từng ảnh một (Single Inference). Chưa hỗ trợ tải lên hàng loạt (Batch Processing) để xử lý nhiều ảnh cùng lúc.

- **Chưa kiểm soát đầu vào chặt chẽ:** Nếu người dùng tải lên ảnh không phải khuôn mặt (ví dụ: ảnh phong cảnh, đồ vật), mô hình vẫn cố gắng dự đoán và đưa ra kết quả sai. Cần tích hợp thêm module phát hiện khuôn mặt (Face Detection) để lọc ảnh trước khi đưa vào mô hình phân loại.

Chương 7
Hạn chế và hướng phát triển

7.1 Hạn chế

Mặc dù hệ thống nhận diện tình trạng hói đầu đã đạt được kết quả khả quan, mô hình vẫn tồn tại một số hạn chế nhất định, chủ yếu xuất phát từ dữ liệu, kiến trúc mạng và phạm vi triển khai của đồ án.

- **Độ nhạy với điều kiện chụp ảnh:** Mô hình có xu hướng giảm độ chính xác khi ảnh đầu vào có ánh sáng yếu, độ tương phản thấp, góc chụp không chính diện hoặc khuôn mặt bị che khuất bởi tóc, mũ, kính hoặc các vật thể khác. Nguyên nhân chủ yếu do dữ liệu huấn luyện chưa bao phủ đầy đủ các tình huống này.

- **Ảnh hưởng của mất cân bằng dữ liệu:** Tập dữ liệu huấn luyện có sự chênh lệch giữa số lượng ảnh thuộc hai lớp *Bald* và *NotBald*. Nếu không có biện pháp xử lý phù hợp, mô hình có thể bị thiên lệch về lớp chiếm đa số, dẫn đến độ chính xác tổng thể cao nhưng hiệu năng kém trên lớp thiểu số.

- **Kiến trúc mạng còn đơn giản:** Mô hình CNN được xây dựng với số lượng lớp tích chập hạn chế và tầng fully-connected có số lượng tham số lớn. Điều này có thể gây ra hiện tượng *overfitting*, đặc biệt khi kích thước tập dữ liệu không đủ lớn.

- **Chưa khai thác được đặc trưng cấp cao:** Do không sử dụng các mô hình học chuyển giao (transfer learning), mạng chưa tận dụng được các đặc trưng trừu tượng đã được học từ các bộ dữ liệu lớn, dẫn đến khả năng tổng quát hóa còn hạn chế.

- **Phạm vi đánh giá còn hạn chế:** Việc đánh giá mô hình chủ yếu dựa trên độ chính xác (accuracy), chưa phản ánh đầy đủ hiệu năng trong các trường hợp dữ liệu mất cân bằng hoặc yêu cầu phân tích chi tiết theo từng lớp.

7.2 Hướng phát triển

Trong tương lai, hệ thống có thể được mở rộng và cải tiến theo nhiều hướng nhằm nâng cao độ chính xác, tính ổn định và khả năng ứng dụng thực tế.

- **Tách riêng transform cho Train và Validation/Test:** Áp dụng các kỹ thuật tăng cường dữ liệu (data augmentation) như xoay, lật, thay đổi độ sáng chỉ cho tập huấn luyện, trong khi giữ nguyên transform cho tập validation và test nhằm đảm bảo kết quả đánh giá phản ánh đúng năng lực thực tế của mô hình.

- **Bổ sung các chỉ số đánh giá nâng cao:** Ngoài accuracy, cần sử dụng thêm các chỉ số như Confusion Matrix, Precision, Recall và F1-score để đánh giá chi tiết hiệu năng mô hình trên từng lớp, đặc biệt trong bối cảnh dữ liệu không cân bằng.

- **Áp dụng học chuyển giao (Transfer Learning):** Thử nghiệm các backbone pretrained như ResNet, MobileNet hoặc EfficientNet và thực hiện fine-tuning nhằm tận dụng tri thức đã học từ các bộ dữ liệu lớn, từ đó cải thiện độ chính xác và khả năng tổng quát hóa của mô hình.

- **Xử lý mất cân bằng dữ liệu:** Áp dụng các kỹ thuật như gán trọng số cho từng lớp (class weighting), sử dụng Focal Loss hoặc các phương pháp lấy mẫu lại dữ liệu (oversampling/undersampling) để giảm thiên lệch trong quá trình huấn luyện.

- **Ước lượng mức độ hói theo xác suất:** Thay vì chỉ đưa ra nhãn nhị phân, hệ thống có thể mở rộng để cung cấp xác suất dự đoán hoặc mức

độ hói đầu theo phần trăm, giúp kết quả trở nên trực quan và hữu ích hơn cho người dùng.

- **Mở rộng triển khai ứng dụng:** Hệ thống có thể được tích hợp hoàn chỉnh vào một ứng dụng web hoặc di động, hỗ trợ người dùng tải ảnh, xem kết quả dự đoán và lưu trữ lịch sử, qua đó tăng tính ứng dụng thực tế của mô hình.

Chương 8

Kết luận và Hướng phát triển

8.1 Tổng kết các kết quả đạt được

Đồ án đã hoàn thành mục tiêu xây dựng một hệ thống *End-to-End* (từ dữ liệu đến ứng dụng) cho bài toán nhận diện đầu hói. Các đóng góp chính của đồ án bao gồm:

1. **Về mặt lý thuyết:** Đã hệ thống hóa được kiến thức nền tảng về Học sâu (Deep Learning) và Mạng nơ-ron tích chập (CNN). Báo cáo đã làm rõ vai trò của các thành phần cốt lõi như lớp tích chập (trích xuất đặc trưng), hàm kích hoạt ReLU (xử lý phi tuyến) và các thuật toán tối ưu hóa hiện đại như Adam.

2. **Về mặt mô hình:** Đã thiết kế và huấn luyện thành công một kiến trúc CNN tùy chỉnh với 3 khối tích chập. Mô hình chứng minh được khả năng hội tụ tốt thông qua hàm mất mát Cross-Entropy và học được các đặc trưng phân biệt giữa hai lớp **Bald** và **NotBald** từ dữ liệu ảnh thô.

3. **Về mặt ứng dụng thực tiễn:** Đã xây dựng thành công giao diện web tương tác sử dụng *Streamlit*. Ứng dụng cho phép người dùng tải ảnh và nhận kết quả dự đoán theo thời gian thực, chứng minh tính khả thi khi triển khai các mô hình học máy vào thực tế đời sống.

8.2 Đánh giá tổng quan

Thông qua quá trình thực nghiệm và đánh giá, nhóm thực hiện rút ra các kết luận sau:

- **Ưu điểm:** Quy trình xây dựng hệ thống mạch lạc, code được tổ chức rõ ràng theo chuẩn PyTorch. Mô hình hoạt động ổn định trên các ảnh chân dung có điều kiện ánh sáng và góc chụp tiêu chuẩn.
- **Tồn tại:** Do giới hạn về tài nguyên tính toán và độ phức tạp của kiến trúc tự xây dựng, mô hình vẫn gặp khó khăn với các "ca khó"(edge cases) như người đội mũ, tóc màu sáng trùng nền, hoặc các trường hợp hói nhẹ (hói chữ M). Ngoài ra, sự mất cân bằng dữ liệu trong tập CelebA cũng ảnh hưởng nhất định đến độ nhạy (Recall) của lớp Bald.

8.3 Hướng phát triển trong tương lai

Để nâng cao hiệu năng và tính ứng dụng của hệ thống, nhóm đề xuất các hướng cải tiến sau:

- **Áp dụng Transfer Learning:** Thay thế kiến trúc tự xây dựng bằng các *Backbone* mạnh mẽ đã được huấn luyện trước như ResNet-50, VGG-16 hoặc MobileNetV2. Việc này giúp tận dụng các đặc trưng đã học từ ImageNet để tăng độ chính xác và giảm thời gian huấn luyện.
- **Xử lý mất cân bằng dữ liệu:** Áp dụng các kỹ thuật chuyên sâu như *Focal Loss* (để mô hình tập trung vào các mẫu khó) hoặc phương pháp *Oversampling/Undersampling* để cân bằng lại tỷ lệ giữa hai lớp.
- **Tích hợp Module phát hiện khuôn mặt:** Bổ sung thêm một bước tiền xử lý sử dụng Haar Cascade hoặc MTCNN để tự động cắt (crop) vùng khuôn mặt trước khi đưa vào mô hình phân loại. Điều này giúp loại bỏ nhiễu nền và tăng độ chính xác của dự đoán.

8.4 Lời kết

Tóm lại, đồ án "Hệ thống nhận biết hói đầu"là bước khởi đầu quan trọng giúp nhóm sinh viên nắm vững quy trình giải quyết một bài toán thị giác máy tính. Mặc dù còn những hạn chế nhất định, nhưng hệ thống đã tạo ra tiền đề vững chắc cho việc nghiên cứu và phát triển các ứng dụng phân tích thuộc tính khuôn mặt phức tạp hơn trong tương lai.

Tài liệu tham khảo

[1] Y. LeCun, Y. Bengio, G. Hinton, *Deep Learning*, Nature, 2015.

[2] Z. Liu, P. Luo, X. Wang, X. Tang, *Deep Learning Face Attributes in the Wild*, ICCV, 2015.

[3] PyTorch Documentation, `https://pytorch.org/docs/stable/index.html`.

[4] TorchVision Documentation, `https://pytorch.org/vision/stable/index.html`.

[5] Streamlit Documentation, `https://streamlit.io`.